

Grafos - Algoritmos III

Joaquin Arango

May 2026

Ejercicio 1: *Equilibrio Digrafo*

Demostrar, usando inducción en la cantidad de aristas, que todo digrafo D satisface:

$$\sum_{v \in V(D)} d_{in}(v) = \sum_{v \in V(D)} d_{out}(v) = |E(D)|$$

Solución

Haremos inducción sobre $n = |E|$

$P(n)$ = todo digrafo D satisface $\sum_{v \in V(D)} d_{in}(v) = \sum_{v \in V(D)} d_{out}(v) = |E(D)|$

Caso Base: $n = 0$

Esto significa que si tomamos $V = \{v_1, \dots, v_m\}$ luego se tiene que $\forall i, j \quad 1 \leq i, j \leq m \quad i \neq j$ entonces $(v_i, v_j) \notin E$.

Luego, en definitiva, $d_{in}(v_i) = d_{out}(v_i) = 0 \quad \forall 1 \leq i \leq m$

$$\therefore \sum_{v \in V(D)} d_{in}(v) = \sum_{v \in V(D)} d_{out}(v) = 0 = |E(D)| \quad \checkmark$$

Caso Base: $n = 1$

Sea $V = \{v_1, \dots, v_m\}$. Esto quiere decir que $\exists!(v_i, v_j) \in E$ con $1 \leq i, j \leq m \quad i \neq j$

Y luego, $\sum_{v \in V(D)} d_{in}(v) = d_{in}(v_j) = 1$ y $\sum_{v \in V(D)} d_{out}(v) = d_{out}(v_i) = 1$

Concluyendo así que: $\sum_{v \in V(D)} d_{in}(v) = \sum_{v \in V(D)} d_{out}(v) = 1 = |E(D)| \quad \checkmark$

Hipótesis Inductiva: supongamos que se cumple $P(j)$ para todo $j \leq n$

Paso inductivo: veamos que se cumple $P(n+1)$

Sea D un grafo arbitrario con $|E_D| = n+1$, y $V_D = \{v_1, \dots, v_m\}$

Sea entonces i', j' con $1 \leq i', j' \leq m$ con $i' \neq j'$ tal que $(v_{i'}, v_{j'}) \in E$.

Defino luego $\tilde{D}$ al digrafo tal que $V_{\tilde{D}} = \{v_1, \dots, v_m\}$, $E_{\tilde{D}} = E \setminus \{(v_{i'}, v_{j'})\}$

Por hipótesis, dado que $|\tilde{E}| = n$, entonces $\sum_{v \in V_{\tilde{D}}} d_{in_{\tilde{D}}}(v) = \sum_{v \in V_{\tilde{D}}} d_{out_{\tilde{D}}}(v) = |\tilde{E}|$

Nótese que $V_D = V_{\tilde{D}}$, $E_D = E_{\tilde{D}} \sqcup \{(v_{i'}, v_{j'})\}$

Y luego, si llamo D^* al grafo tal que $V_{D^*} = V_D$ y $E_{D^*} = \{(v_{i'}, v_{j'})\}$

Como vimos en el caso base, dado que $|E_{D^*}| = 1$, entonces

$$\sum_{v \in V_{D^*}} d_{in_{D^*}}(v) = d_{in_{D^*}}(v_{j'}) = 1 \quad \text{y} \quad \sum_{v \in V_{D^*}} d_{out_{D^*}}(v) = d_{out_{D^*}}(v_{i'}) = 1$$

En conjunto, tenemos luego que

$$\begin{aligned} \sum_{v \in V(D)} d_{in}(v) &= \sum_{v \in V_{\tilde{D}}} d_{in_{\tilde{D}}}(v) + \sum_{v \in V_{D^*}} d_{in_{D^*}}(v) = \sum_{v \in V_{\tilde{D}}} d_{out_{\tilde{D}}}(v) + \sum_{v \in V_{D^*}} d_{out_{D^*}}(v) \\ &= \sum_{v \in V(D)} d_{out}(v) = |\tilde{E}| + |E_{D^*}| = |E_D| = n+1 \quad \text{como se quería ver.} \end{aligned}$$

Luego, como D era arbitrario, funciona para todo D con $|E| = n+1$

Finalmente, por principio de inducción, $P(n)$ vale $\forall n \in \mathbb{N}$



Ejercicio 2: *Doble Grado*

Demostrar, usando la técnica de reducción al absurdo, que todo grafo no trivial tiene al menos dos vértices del mismo grado. Ayuda: prestar atención a la secuencia ordenada de los grados de los vértices.

Solución

Supongamos que no es cierto. Es decir, planteemos que existe un grafo no trivial D con $V = \{v_1, \dots, v_n\}$ tal que $d(v_i) \neq d(v_j) \quad \forall 1 \leq i, j \leq n$ con $i \neq j$

Ahora bien, para cada v_i , ¿cuál es valor mínimo y máximo que puede tomar $d(v_i)$?

Claramente, $d(v_i) \geq 0$; y, por otro lado, la cantidad máxima de aristas que puede tener una arista de un grafo va a equivaler a la cantidad de aristas resultante de estar conectado con todo el resto de los nodos. Es decir, $d(v_i) \leq n-1$.

En conjunto, $0 \leq d(v_i) \leq n-1 \quad \forall 1 \leq i \leq n$.

Sin embargo, hay un pequeño detalle: si $\exists 1 \leq j \leq n : d(v_j) = n-1$ entonces $\nexists 1 \leq k \leq n : d(v_k) = 0$ (en palabras: no puede haber un nodo que está conectado con todosz uno que "no está conectado a ninguno" simultáneamente).

Se desprende entonces que:

- Si $\exists 1 \leq j \leq n : d(v_j) = n - 1 \Rightarrow 1 \leq d(v_i) \leq n - 1 \quad \forall 1 \leq i \leq n$

Por lo tanto, eso quiere decir que tengo n vértices y $n - 1$ grados distintos.

Por principio del palomar, necesariamente $\exists 1 \leq i, j \leq n$ con $i \neq j$ tal que $d(v_i) = d(v_j)$ *ABS* pues habíamos supuesto que $d(v_i) \neq d(v_j) \quad \forall 1 \leq i, j \leq n$ con $i \neq j$.

- Si $\exists 1 \leq j \leq n : d(v_j) = 0 \Rightarrow 0 \leq d(v_i) < n - 1 \quad \forall 1 \leq i \leq n$

Por lo tanto, eso quiere decir que tengo n vértices y $n - 1$ grados distintos.

Por principio del palomar, necesariamente $\exists 1 \leq i, j \leq n$ con $i \neq j$ tal que $d(v_i) = d(v_j)$ *ABS* por lo mismo que antes.

Y el absurdo vino de suponer que existía D grafo no trivial cuyos vértices poseen todos grados distintos.

Entonces, para todo grafo D no trivial, existen al menos dos vértices con mismo grado como se quería probar.



Ejercicio 3: *Unicidad Digrafo*

Un grafo orientado es un digrafo D tal que al menos uno de $v \rightarrow w$ y $w \rightarrow v$ no es una arista de D , para todo $v, w \in V(D)$. En otras palabras, un grafo orientado se obtiene a partir de un grafo no orientado dando una dirección a cada arista.

Demostrar en forma constructiva que para cada n existe un único grafo orientado cuyos vértices tienen todos grados de salida distintos. Ayuda: aprovechar el ejercicio anterior y observar que el absurdo no se produce para un único grafo orientado

Solución

Construiremos cada solución sobre $n = |V|$

$P(n)$ = existe un único grafo orientado de n vértices cuyos grados de salida son todos distintos.

Aclaración previa: un grafo $D = (V, E)$ será *distinto* de otro $\tilde{D} = (\tilde{V}, \tilde{E})$ si $\nexists f : V \rightarrow \tilde{V}$ tal que para todo $v, w \in V$: $(v, w) \in E \iff (f(v), f(w)) \in \tilde{E}$
Es decir, si $D, \tilde{D}$ no son *isomorfos*.

Caso Base: $n=1$ Trivial

Caso Base: $n = 2$

Sea $D = (V, E)$ un grafo orientado. Luego, como $|V| = n = 2$, se tiene que $V = \{v_1, v_2\}$. Por lo visto en el anterior ejercicio, $0 \leq d(v_1), d(v_2) \leq 1$. Luego, como requiero que $d(v_1) \neq d(v_2)$, tomo $d(v_1) = 0 \wedge d(v_2) = 1$. Es decir, $E = \{(v_2, v_1)\}$



Figura 1: Grafo definido

Nótese además que yo podría haber definido $d(v_1) = 1 \wedge d(v_2) = 0$. Luego, si llamo $\tilde{D} = (V, \tilde{E})$ con $\tilde{E} = \{(v_1, v_2)\}$ entonces $D, \tilde{D}$ son isomorfos.



Figura 2: Grafo isomorfo

Luego, no hay otra forma de asignar los grados sin que haya repetición de ellos.

Caso Base: $n = 3$

Sea $D = (V, E)$ un grafo orientado de tres vértices. Es decir, $V = \{v_1, v_2, v_3\}$. Luego, $0 \leq d(v_1), d(v_2), d(v_3) \leq 2$.

Por lo tanto, tengo tres opciones de grados diferentes, y tres nodos. Hay varias formas isomorfas de armar las conexiones para que cada vértice posea un grado distinto al resto. Miremos tres de ellas.

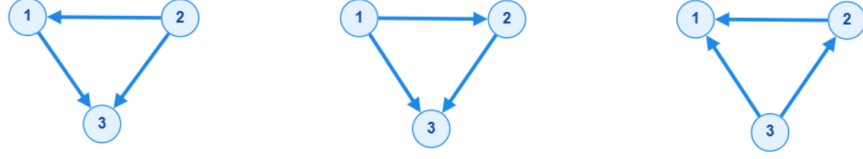


Figura 3: Grafos isomorfos de tres vértices

Es decir, la manera de construir el grafo orientado es a partir de asignar a cada vértice v_j una cantidad i de conexiones con $0 \leq i, j \leq 2$ pero siendo cuidadosos de que si $(v_i, v_k) \in E \Rightarrow (v_k, v_i) \notin E$.

Para este caso, podemos proponer $D = (V, E)$ con $V = \{v_1, v_2, v_3\}$, $E = \{(v_2, v_1), (v_3, v_1), (v_3, v_2)\}$ (ver Figura 3, grafo derecho).

Y este grafo es único (salvo isomorfismo) puesto que si tengo otro grafo orientado $\tilde{D}$ de tres vértices con $d(v_i) = 0$, $d(v_j) = 1$, $d(v_k) = 2$ con $i, j, k \in \{1, 2, 3\}$ distintos, entonces v_k apunta tanto a v_i como a v_j ; v_j solo puede apuntar a v_i (pues su grado es 1, y v_k necesariamente lo está apuntando; entonces no puede darse simultáneamente $v_j \rightarrow v_k$) y v_i no apunta a ninguno. Luego, es claro que se trata de una permutación $f : V_D \rightarrow V_{\tilde{D}}$ tal que

$$f(v_1) = f(v_i); f(v_2) = f(v_j); f(v_3) = f(v_k).$$

Es decir, $D, \tilde{D}$ son isomorfos.

Caso general: quiero ver que se cumple $P(n+1)$

Sea luego $V = \{v_1, \dots, v_n, v_{n+1}\}$, $E = \{(v_i, v_j) : 1 \leq j < i \leq n+1\}$

Veamos que $D = (V, E)$ es un grafo orientado.

Sea $1 \leq i \leq n+1$. Luego, $(v_i, v_j) \in E \quad \forall 1 \leq j \leq i$.

Por lo tanto, $d(v_i) = i-1$ (y luego $0 \leq d(v_i) \leq n$) $\forall 1 \leq i \leq n+1$.

Eso quiere decir que $d(v_i) = d(v_k) \iff i-1 = k-1 \iff i = k$

Así se tiene que $d(v_i) \neq d(v_k) \quad \forall 1 \leq i, k \leq n+1 \quad i \neq k$

Por otro lado, si $i > k \Rightarrow (v_i, v_k) \in E$ pero $(v_k, v_i) \notin E$.

Luego, D es un grafo orientado de $n+1$ vértices.

Y, si existe otro grafo orientado $\tilde{D}$ de $n+1$ vértices; entonces el vértice de grado n está conectado a todos; el de grado $n-1$ está conectado a todos los vértices excepto el de grado n ; el de grado $n-2$, está conectado al resto de vértices (es decir, exceptuando los que presentan grado n y $n-1$); y así sucesivamente.

Luego, si llamamos $v_{\tilde{j}_r}$ al vértice de $\tilde{D}$ que presenta grado r con $0 \leq r \leq n$, entonces puedo armar $f : V_D \rightarrow V_{\tilde{D}}$ tal que: $f(v_i) = f(v_{\tilde{j}_{i-1}})$.

Dado que f biyectiva (pues para cada grado $i-1$ existe único $v_{\tilde{j}}$ con $d(v_{\tilde{j}}) = i-1$ puesto que $\tilde{D}$ grafo orientado) entonces $D, \tilde{D}$ son isomorfos.

Por lo tanto, D existe y es único (salvo isomorfismo).

Finalmente, por construcción, $P(n)$ vale $\forall n \in \mathbb{N}$ como se quería ver.



Ejercicio 4: *Arte Conexo*

Un vértice v de un grafo G es un *punto de articulación* si $G - v$ tiene más componentes conexas que G . Por otro lado, un grafo es *biconexo* si es conexo y no tiene puntos de articulación.

a) Demostrar, usando inducción en la cantidad de vértices, que todo grafo de n vértices que tiene más de $(n-1)(n-2)/2$ aristas es conexo.

b) Demostrar por medio de una reducción al absurdo que todo grafo de n vértices que tenga al menos $2 + (n-1)(n-2)/2$ aristas es biconexo.

Solución a)

Haremos inducción sobre $n = |V|$

$P(n)$ = todo grafo de n vértices que tiene más de $(n-1)(n-2)/2$ aristas es conexo.

Caso Base: $n=1$ Trivial

Caso Base: $n = 2$

Sea $V = \{v_1, v_2\}$, E tal que $|E| > 0 = (2-1)(2-2)/2$

Defino $G = (V, E)$. Veamos que G es conexo.

Como $|E| > 0 \Rightarrow |E| = 1$. Luego, eso quiere decir que: $(v_1, v_2) \in E$

Por lo tanto, dado que existe camino entre todo par de vértices (pues $V = \{v_1, v_2\}$ y se tiene que $(v_1, v_2) \in E$) entonces G es conexo.

Y como G era un grafo de dos vértices con al menos una arista arbitrario, entonces todo grafo G con dos vértices que tiene más de $0 = (2-1)(2-2)/2$ aristas es conexo ✓.

Hipótesis Inductiva: supongamos que se cumple $P(n)$.

Paso inductivo: veamos que se cumple $P(n+1)$

Sea G un grafo arbitrario de $n+1$ vértices ($V = \{v_1, \dots, v_n, v_{n+1}\}$) con $|E| > n(n-1)/2$.

Sea luego $v \in V$ tal que $d(v) < n-1$

Nótese que, si $\exists v \in V$ tal que $d(v) = n$ entonces G es conexo, pues v está conectado a todos los vértices, y por lo tanto $\forall 1 \leq i \neq j \leq n+1$ existe camino

entre v_i y v_j ya que es suficiente tomar $v_i \rightarrow v$ (suponiendo $v_i \neq v$) y luego $v \rightarrow v_j$ (suponiendo $v \neq v_j$).

Por otro lado, si $d(v) = n - 1 \quad \forall v \in V$ entonces si tomo $v_i, v_j \in V$ ($i \neq j$) entonces o bien $(v_i, v_j) \in E$ o luego $\exists v_k \in E$ tal que $(v_i, v_k) \in E \wedge (v_k, v_j) \in E$ (puesto que de lo contrario existiría $v_k \in V$ tal que $d(v_k) < n - 1$), por lo que para todo par de vértices de V hay un camino; y entonces G es conexo.

Veamos ahora el caso $v \in V$ tal que $d(v) < n - 1$.

Sea $\tilde{V} = V \setminus \{v\}$. Luego, $|\tilde{V}| = n$. Por otro lado, si tomamos

$\tilde{E} = \{(v_i, v_j) \in E : v_i \neq v \wedge v_j \neq v\}$ entonces se tiene que:

$$|\tilde{E}| = |E| - d_G(v) > n(n-1)/2 - (n-1) = (n-1)(n-2)/2$$

Por lo tanto, por hipótesis inductiva, $\tilde{G}$ es conexo.

Y luego, como $|E| > n(n-1)/2$ entonces dado que $|\tilde{E}| \leq n(n-1)/2$ (es decir, $\tilde{E}$ es a lo sumo completo), entonces:

$$d_G(v) \geq |E| - |\tilde{E}| > n(n-1)/2 - n(n-1)/2 = 0 \Rightarrow d_G(v) > 0$$

Luego, G es conexo, puesto que $\forall v_i, v_j \in V$ tal que $v_i \neq v_j$, entonces si $v_i, v_j \in \tilde{V}$, por hipótesis existe camino entre ellos; y, por otro lado, si $v_i = v$, entonces o bien $(v, v_j) \in E$ o luego, como $d_G(v) > 0$, entonces $\exists v_k \in V$ tal que $(v, v_k) \in E$ y dado que $v_k, v_j \in \tilde{V}$, entonces existe camino entre ellos; por lo que, agrupando, tenemos que existe camino entre v, v_j (caso análogo, $v_j = v$).

Y como G era un grafo de $n+1$ vértices con más de $n(n-1)/2$ aristas arbitrario, entonces todo grafo G de $n+1$ vértices y más de $n(n-1)/2$ aristas es conexo (cumpliéndose así $P(n+1)$).

Finalmente, por principio de inducción, $P(n)$ vale $\forall n \in \mathbb{N}$.



Solución b)

Supongamos que no es cierto. Es decir, supongamos que existe $n \in \mathbb{N}_{>2}$ (caso $n \in \{1, 2\}$ trivial) tal que se observa un grafo G de n vértices y al menos $2 + (n-1)(n-2)/2$ aristas que no es biconexo.

Es decir, $\exists v \in V$ tal que v es un punto de articulación ($G-v$ tiene más componentes conexas que G).

Luego, notar que, por inciso a), G es conexo. Luego, G tiene una única componente conexa (que es la misma G). Además, si tomo $\tilde{V} = V \setminus \{v\}$, $\tilde{E} = \{(v_i, v_j) \in E : v_i \neq v \wedge v_j \neq v\}$ entonces $\tilde{G} = (\tilde{V}, \tilde{E})$ es conexo pues:

$$|\tilde{E}| = |E| - d_G(v) \geq 2 + (n-1)(n-2)/2 - (n-1) = (4+n^2-2n-n+2-2n+2)/2 = (n^2-5n+8)/2 > (n^2-5n+6)/2 = (n-2)(n-3)/2$$

Es decir que $|\tilde{E}| > (n-2)(n-3)/2$

Y como $\tilde{G}$ es un grafo de $n-1$ vértices y más de $(n-2)(n-3)/2$ aristas, entonces nuevamente por inciso a), $\tilde{G}$ es conexo.

Pero entonces, $\tilde{G}$ también tiene una única componente conexa (que es la misma $\tilde{G}$).

Por lo tanto, $G, \tilde{G}$ tienen la misma cantidad de componentes conexas ABS

Y el absurdo vino de suponer que existe G de n vértices y al menos $2 + (n - 1)(n - 2)/2$ aristas que no es biconexo.
 $\therefore$ todo grafo de n vértices que tenga al menos $2 + (n - 1)(n - 2)/2$ aristas es biconexo como se quería probar.



Ejercicio 5: *Ciclo Compartido*

Sean P y Q dos caminos distintos de un grafo G que unen un vértice v con otro w . Demostrar en forma directa que G tiene un ciclo cuyas aristas pertenecen a P o Q . Ayuda: denotar $P = \{v_0, \dots, v_p\}$ y $Q = \{w_0, \dots, w_q\}$ con $v_0 = w_0 = v$ y $v_p = w_q = w$. Definir explícitamente cuáles son los subcaminos de P y Q cuya unión forman un ciclo.

Solución

Sea luego $P = \{v_0, \dots, v_p\}$, $Q = \{w_0, \dots, w_q\}$ dos caminos distintos que unen a v con w , donde $v, w \in V_G$.

Luego, $v = v_0 = w_0$, $w = v_p = w_q$

Ahora bien, como $P \neq Q$ y $v = v_0 = w_0$, entonces $\exists 1 \leq i < \min\{p, q\}$ tal que $v_i \neq w_i$. Luego, en particular podemos tomar i como la primer instancia en el que eso sucede. Es decir, tomo $1 \leq i < \min\{p, q\}$ tal que $v_k = w_k \quad \forall 0 \leq k < i$ y $v_i \neq w_i$.

Pero además, dado que $w = v_p = w_q$ entonces $\exists i < j_P \leq p, i < j_Q \leq q$ tal que $v_{j_P} = w_{j_Q}$ (en particular, bien podría ser $j_P = p, j_Q = q$). Luego, por principio de buena ordenación, existen j_P, j_Q tal que $v_{j_k} \notin Q \quad \forall i < k < j_P$ y $v_{j_P} \in Q$, es decir, $v_{j_P} = w_{j_Q}$ con $i < j_P \leq p, i < j_Q \leq q$.

Sea entonces $C = \underbrace{v_{i-1} v_i \dots v_{j_P-1}}_{w_{i-1}} \underbrace{v_{j_P} w_{j_Q-1} \dots w_i}_{w_{j_Q}} \underbrace{w_{i-1}}_{v_{i-1}}$. Veamos que C es un ciclo.

En primer lugar, C empieza y finaliza en el mismo vértice por definición de i .

Por otro lado, C tiene al menos tres vértices, pues $i - 1 < i < j_P, j_Q$ (y por lo tanto, $v_{i-1} \neq v_i, v_i \neq v_{j_P}, v_{i-1} \neq v_{j_P}$)

Como P es camino, entonces $v_{i-1}, v_i, \dots, v_{j_P}$ son distintos dos a dos. Por otro lado, como Q es camino, $w_{j_Q}, w_{j_Q-1}, \dots, w_i, w_{i-1}$ son distintos dos a dos.

Además, por definición j_P, j_Q , se tiene que $\forall v \in \{v_i, \dots, v_{j_P-1}\}, v \notin \{w_{j_Q-1}, \dots, w_i\}$ y viceversa. Por lo tanto, como tengo un circuito (pues es un recorrido que empieza y termina en el mismo vértice) con más de tres vértices que no pasa por un mismo vértice (pues todos son distintos entre sí), entonces existe un ciclo C entre v y w en mi grafo G como se quería probar.



Ejercicio 7: Intersección máxima

Sea G un grafo conexo. Demostrar por el contrarrecíproco que todo par de caminos simples de longitud máxima de G tienen un vértice en común. Ayuda: suponer que hay dos caminos disjuntos en vértices de igual longitud y definir explícitamente un camino que sea más largo que ellos.

Solución

Como dice la sugerencia, si G es un grafo conexo, ver que todo par de caminos simples de longitud máxima de G tienen un vértice común es equivalente a ver que si tengo dos caminos disjuntos de misma longitud, entonces es posible definir un camino de mayor longitud que ellos.

Sea entonces P, Q dos caminos disjuntos de G , ambos de longitud l .

Es decir, $P = \{p_0, p_1, \dots, p_l\}, Q = \{q_0, q_1, \dots, q_l\}$ donde $P, Q \subseteq V$ y $P \cap Q = \emptyset$

Como G conexo, yo sé que $\exists C = \{p_i, c_0, \dots, c_n, q_j\}$ camino entre p_i y q_j

$\forall 0 \leq i, j \leq l$.

Si $(p_i, q_l) \in E_G$ entonces si defino $R = \{p_0, \dots, p_i, q_l, \dots, q_0\}$ entonces

$I(R) = i + l > I(P) = I(Q) = l$ (ídem caso q_j y p_l).

En definitiva, como P, Q disjuntos, yo sé que $\exists i, j$ tal que $C = \{p_i, c_0, \dots, c_n, q_j\}$

cumple que C es camino y $c_k \notin P \wedge c_k \notin Q \quad \forall 0 \leq k \leq n$.

Esto se cumple pues G conexo, y luego puedo tomar por ejemplo $\tilde{C} = \{p_l, c_0, \dots, c_m, q_l\}$

y, a partir de éste, tomar $C \subseteq \tilde{C}$ el camino entre $\tilde{p}, \tilde{q}$ donde, si $\exists 0 \leq k \leq n$ tal que $c_k \in P$, entonces $\tilde{p} = c_k$ con $c_i \notin P \quad \forall k < i \leq n$; o luego $\tilde{p} = p_l$; y si $\exists 0 \leq r \leq n$ tal que $c_r \in Q$, entonces $\tilde{q} = c_r$ con $c_i \notin Q \quad \forall r < i \leq n$; o luego $\tilde{q} = q_l$. En síntesis, tomo C camino mínimo entre P y Q .

Habiendo probado esto, entonces basta con notar que si tomo $C = \{p_i, c_0, \dots, c_n, q_j\}$ camino entre un p_i y un q_j tal que $c_k \notin P \wedge c_k \notin Q \quad \forall 0 \leq k \leq n$, entonces puedo armar un camino R de la siguiente forma:

Nótese que si tomo $r_P = \max\{i, l - i\}$ la mayor distancia que se puede recorrer en el camino P para llegar a p_i , entonces $r_P \geq l/2$.

Ídem con $r_Q = \max\{j, l - j\}$, $r_Q \geq l/2$.

SPG, supongamos que $i \geq l/2$ (es decir, $r_P = i$) y $j \leq l/2$, es decir, $r_Q = l - j$ (los otros casos son completamente análogos).

Es así que, tomando $R = \{ \underbrace{p_1, \dots, p_i}_{I(\{p, \dots, p_i\}) \geq l/2}, \underbrace{c_0, \dots, c_n}_{I(C) > 0}, \underbrace{q_j, \dots, q_l}_{I(\{q_j, \dots, q_l\}) \geq l/2} \}$,

$$I(\{p, \dots, p_i\}) \geq l/2 \quad I(C) > 0 \quad I(\{q_j, \dots, q_l\}) \geq l/2$$

entonces $I(R) > l$.

Luego, como a partir de dos caminos disjuntos de longitud l de un grafo conexo G cualquiera, logré hallar un camino de mayor longitud, entonces eso significa que si tengo un grafo conexo, luego para todo par de caminos disjuntos de igual longitud del mismo, es posible definir otro camino que posea una mayor longitud. Por lo tanto, dado G grafo conexo, todo par de caminos simples de longitud máxima de G tienen un vértice en común como se quería probar.



Ejercicio 14: *Representa Grafos*

Discutir (brevemente) las ventajas y desventajas en cuanto a la complejidad temporal y espacial de las siguientes implementaciones de un conjunto de vecindarios para un grafo G , de acuerdo a las siguientes operaciones:

Operaciones:

- a) Inicializar la estructura a partir de un conjunto de aristas de G .
- b) Determinar si dos vértices v y w son adyacentes.
- c) Recorrer y/o procesar el vecindario $N(v)$ de un vértice v dado.
- d) Insertar un vértice v con su conjunto de vecinos $N(v)$.
- e) Insertar una arista vw .
- f) Remover un vértice v con todas sus adyacencias.
- g) Remover una arista vw .
- h) Mantener un orden de $N(v)$ de acuerdo a algún invariante que permita recorrer cada vecindario en un orden dado.

Solución en función de la estructura de datos

1. N se representa con una secuencia (vector o lista) que en cada posición v tiene el conjunto $N(v)$ implementado sobre una secuencia (lista o vector). Cada vértice es una estructura que tiene un índice para acceder en $O(1)$ a $N(v)$. Esta representación se conoce comúnmente como *lista de adyacencias*.

Al momento de inicializar la estructura, primero debo generar un vector de tamaño $|V|$, es decir, en primera instancia tengo un costo espacial y temporal $O(|V|)$. Luego, para cada par (v_i, v_j) de E , debo acceder a la posición v_i (en $O(1)$) e insertar al final de la lista asociada el elemento v_j (lo que también cuesta $O(1)$).

El costo espacial y temporal final es entonces $O(|V| + |E|)$, o análogamente, $O(\max(|V|, |E|))$.

Por otro lado, para determinar si dos vértices v y w son adyacentes, debo empezar accediendo en $O(1)$ a $N(v)$ (o, análogamente, a $N(w)$) y recorrer su lista de vecinos hasta lograr determinar si w se encuentra en ella. Es decir, la complejidad temporal es de $O(|N(v)|)$ (o bien $O(|N(w)|)$ si recorro los vecinos de w en busca de determinar si v se encuentra entre ellos).

De forma similar al caso anterior, si quiero recorrer y/o procesar el vecindario $N(v)$ de un vértice v dado, esto se realiza en un costo temporal $O(|N(v)|)$ dado que acceder a $N(v)$ es $O(1)$ y recorrerlo cuesta $O(|N(v)|)$.

Si quiero ahora insertar un vértice v con su conjunto de vecinos $N(v)$, entonces en principio debo crear un nuevo vector de vértices (lo que cuesta $O(|V|)$) y asignarle a cada vértice anterior su respectiva lista (si lo hago por referencia, un costo $O(1)$ por nodo; si en cambio es copia, en conjunto se trataría de un costo temporal adicional $O(|E|)$).

Luego, habiendo adicionado nuestro nuevo vértice, acceder a $N(v)$ es $O(1)$. Y luego, insertar su vecindario tiene un costo $O(|N(v)|)$ que sale de la creación de una lista de tamaño $|N(v)|$, y de adicionar en $O(1)$ estas mismas conexiones a los vecindarios de los vértices con los que se encuentra conectado.

Por lo tanto, el costo temporal final es de $O(|V| + |E| + |N(v)|)$ (suponiendo que se hizo una copia de cada lista anterior; de lo contrario, la respuesta es $O(|V| + |N(v)|)$).

Al momento de insertar una arista vw , si se sabe de antemano que no existe la arista en nuestra estructura, entonces esto se puede realizar en $O(1)$ dado que basta con acceder en $O(1)$ a $N(v)$, $N(w)$ y agregar al final de sus listas a v y w respectivamente (también en $O(1)$).

Por otro lado, si no se conoce si la arista ya ha sido insertada, entonces es necesario recorrer $N(v)$ (o $N(w)$) para identificar si debo o no hacer la adición. Luego, esto presenta un costo temporal de $O(|N(v)|)$ (o $O(|N(w)|)$).

En cuanto a remover un vértice v con todas sus adyacencias, nuevamente hay dos opciones:

O bien se puede realizar en $O(1)$ si simplemente “olvidas” la información del espacio v en el vector (es decir, asignas un valor *null*).

O luego, se puede redimensionar el vector en $O(|V|)$ y, nuevamente, decidir asignar a cada posición v su vecindario $N(v)$ por referencia en $O(1)$ (consiguiendo un costo temporal final de $O(|V|)$); o bien realizar una copia nueva de cada una de los vecindarios, lo que toma en conjunto $O(|E|)$. Luego, la complejidad temporal es de $O(|V| + |E|)$.

Por otra parte, remover una arista vw presenta un costo temporal $O(|N(v)| + |N(w)|)$ pues se trata de recorrer las listas $N(v)$ y $N(w)$, eliminando los punteros a w y v respectivamente.

Por otro lado, para cada $N(v)$, mantener un orden cuesta $O(|N(v)|)$, puesto que, al momento de insertar, es necesario recorrer la lista para conocer dónde se debe realizar la inserción para mantener el invariante; y, por otro lado, al eliminar una adyacencia, el conocer cierto orden, en el peor caso, tampoco te salva de tener que recorrer la lista para hallar el elemento (o, por lo menos, tener que recorrerla hasta cierto punto) y hacer el cambio de punteros.

2. Ídem anterior, pero cada $w \in N(v)$ se almacena junto con un índice a la posición que ocupa v en $N(w)$. Esta representación también se conoce como lista de adyacencias, pero tiene información para implementar operaciones dinámicas.

Ahora pensaremos en $N(v)$ como un vector de pares ordenados. Nótese que, en caso de implementarlo como lista, luego en el peor caso no hay diferencia en cuanto a los costos temporales y espaciales anteriormente presentados.

Inicializar la estructura a partir de un conjunto de aristas de G , en un inicio, presenta un costo temporal y espacial de $O(|V|)$ para crear el vector de vértices. Luego, debo adicionar además el costo que lleva la construcción de los vectores $N(v)$. Esto presenta un costo amortizado de $O(2|E|) = O(|E|)$. Luego, el costo temporal y espacial de inicializar la estructura es de $O(|V| + |E|)$.

Al momento de determinar si dos vértices son adyacentes, nuevamente requiero recorrer $N(v)$ (o luego, $N(w)$), lo que presenta un costo de $O(|N(v)|)$. Y dado que el acceder a $N(v)$ es $O(1)$, entonces el costo temporal es de $O(|N(v)|)$.

Similarmente al caso anterior, recorrer o procesar un vecindario tiene un costo de $O(|N(v)|)$, pues acceder al mismo cuesta $O(1)$ y recorrerlo $O(|N(v)|)$.

Si se desea insertar un vértice v con su conjunto de vecinos $N(v)$, entonces hay varias cosas a considerar:

Si el vector de vértices está completo, entonces debo realizar una redimensión del mismo, lo que tiene un coste inicial de $O(|V|)$. Luego, si cada vector vecindario $N(v)$ se pasa por referencia, entonces se mantiene el costo temporal anterior. Si, en cambio, se realiza una nueva copia de cada uno de los vecindarios, entonces en conjunto se presenta un coste adicional de $O(|E|)$.

Si en cambio se hace un análisis amortizado, donde el vector de vértices presenta un tamaño tal que no es necesario una redimensión del vector cada vez que se realiza una inserción, entonces simplemente se presenta un costo inicial $O(1)$.

Luego, cualquiera sea el caso, al momento de la creación del vecindario del vértice v insertado debo, para cada vecino w hacer la respectiva inserción en $N(w)$ con la posición en la que se va a insertar en v , y a la vez conocer cuál será la respectiva en w . Si vemos el costo amortizado, esto lleva entonces $O(1)$ por cada vecino w . Es decir, la inserción de $N(v)$ presenta un costo temporal amortizado de $O(|N(v)|)$.

Por lo tanto, bajo un análisis amortizado, entonces el costo temporal final es de $O(|N(v)|)$. De lo contrario, puede ascender a ser mayor a $O(|V| + \sum_{w \in N(v)} d(w))$,

donde el último término sale de, en cada oportunidad de inserción de vecinos, tener que redimensionar el vector asociado a cada vecindario.

Si simplemente se quiere insertar una lista vw , si se realiza un análisis amortizado, el costo es simplemente $O(1)$ si se conoce con anterioridad que no ha sido re-

gistrada, puesto que es simplemente averiguar en $O(1)$ los valores $|N(v)|, |N(w)|$ e insertar al final del vector $N(v)$ el par $(w, |N(w)|)$ y en el vector $N(w)$ el par $(v, |N(v)|)$.

No obstante, si no se conoce si la arista ya ha sido registrada, entonces es necesario recorrer previamente alguno de los vecindarios $N(v), N(w)$ para determinarlo, presentando un costo $O(|N(v)|)$ (o $O(|N(w)|)$).

En cuanto a remover un índice v con todas sus adyacencias, es necesario luego eliminar a cada arista que involucre al vértice v . Esto se puede realizar, para cada $w \in N(v)$, en $O(1)$; puesto que se almacena la información sobre dónde se haya esa arista en $N(v)$, y luego el acceso y borrado se da (amortizadamente) en $O(1)$.

En definitiva, el costo amortizado es de $O(N(v))$.

Y si solo quiere eliminar una arista vw , en principio se debe recorrer solamente uno de los vecindarios ($N(v)$ o $N(w)$), puesto que luego, al encontrar la arista, puedo observar en qué posición del vector que guarda la información del otro vecindario se encuentra la adyacencia a con mi vértice. Es decir que si yo recorro $N(v)$, al momento de hallar el par (w, i) , puedo en $O(1)$ acceder a $N(w)$ y en $O(1)$ acceder a la i ésima posición de $N(w)$, y hacer la remoción (que, bajo un análisis amortizado, cuesta $O(1)$).

Por lo tanto, el costo temporal final es de $O(|N(v)|)$ o bien $O(|N(w)|)$, en función de qué vecindario se decida recorrer.

Finalmente, mantener un orden de $N(v)$ costaría $O(|N(v)|)$, puesto que si bien las búsquedas se pueden realizar en costos logarítmicos $O(\log |N(v)|)$ utilizando algoritmos de búsqueda binaria; al momento de insertar o remover, si bien identificar posiciones es logarítmico, para mantener el invariante se requeriría, en el peor de los casos, desplazar todos los elementos un lugar cada oportunidad, presentando un costo de $O(|N(v)|)$.

3. $N(v)$ se representa con un vector que en cada posición i tiene un vector booleano A_i con $|V(G)|$ posiciones tal que $A_i[j]$ es verdadero si y solo si i es adyacente a j . Esta representación se conoce comúnmente como *matriz de adyacencias*.

Inicializar la estructura a partir de un conjunto de aristas de G presenta un costo temporal y espacial de $O(|V|^2)$ puesto que se trata de, para cada $v \in V$, fabricar un vector booleano de tamaño $|V|$.

Y luego, dado que la asignación de los valores booleanos es $O(1)$ y tengo en total $O(|E|)$ aristas, entonces tengo un costo temporal adicional $O(|E|)$. Luego, la complejidad temporal final es $O(|V|^2 + |E|) = O(|V|^2)$.

Determinar si dos vértices v y w son adyacentes se puede realizar en $O(1)$ puesto que es simplemente revisar si el valor de la posición $A_v[w]$ en la matriz de adyacencia es *True* o *False*.

Recorrer y/o procesar el vecindario $N(v)$ de un vértice v dado presenta un costo $O(|V|)$ puesto que es el costo de recorrer cada celda del vector A_v (el cual tiene dimensión $|V|$).

La complejidad de insertar un vértice v con su conjunto de vecinos $N(v)$ depende de la implementación inicial de la matriz de adyacencia.

Es raro que a una matriz se le agreguen filas o columnas (es una estructura que está pensada más bien para ser inmutable en ese sentido). Sin embargo, si en la implementación ésta fue creada con espacios vacíos con el fin de “adelantarse” a esta situación, entonces se tendrá un costo $O(|V|)$ puesto que se debe asignar el valor *True* a $A_v(w)$ y a $A_w(v)$ para cada $w \in N(v)$; y *False* a $A_v(\tilde{w})$ para cada $\tilde{w} \in V \setminus N(v)$.

En caso de que se requiera redimensionar la matriz, entonces el costo temporal final será de $O(|V|^2)$ puesto que es inicializar una nueva matriz de dimensiones $(|V| + 1) \times (|V| + 1)$ y asignar los valores booleanos correspondientes.

Por otro lado, insertar una arista vw se puede realizar en $O(1)$ pues es simplemente asignar *True* a las celdas $A_v[w]$ y $A_w[v]$.

Ahora bien, remover un vértice v con todas sus adyacencias presenta el mismo problema que con la inserción.

O bien se puede “olvidar” la información de la fila correspondiente a v , y luego en $O(|N(v)|)$ simplemente asignar *False* a las celdas $A_w[v]$ para cada $w \in N(v)$.

De lo contrario, se tendría que crear una nueva matriz de dimensiones $(|V| - 1) \times (|V| - 1)$ y completar en $O(|V|^2)$ la información corresponde de cada celda de la misma (es decir, el costo temporal es de $O(|V|^2)$).

Sin embargo, remover una arista vw se puede realizar en $O(1)$ pues es simplemente asignar *False* a las celdas $A_v[w]$ y $A_w[v]$.

Por último, mantener un orden de $N(v)$ de acuerdo a algún invariante que permita recorrer cada vecindario en un orden dado no es posible (o, mejor dicho, no es viable) en una matriz de adyacencia; puesto que si modifico las posiciones que representan las columnas solo para una fila, necesariamente afectará al resto de la matriz. Luego, esto implicaría que cada vez debería reorganizar en $O(|V|^2)$ la matriz (o directamente crear una nueva) para mantener el orden deseado.

Es decir, en una matriz de adyacencia, donde las posiciones están dadas por el identificador de los vértices v (y recorrer $N(v)$ bajo esta disposición cuesta $O(|V|)$), el orden es global para toda la estructura.

4. N se representa con un vector que en cada posición v tiene el conjunto $N(v)$ implementado con una tabla de hash. Esta representación es un mix entre las representaciones clásicas de matriz de adyacencias y lista de adyacencias.

El costo de inicializar la estructura a partir de un conjunto de aristas de G tiene un costo temporal y espacial $O(|V| + |E|)$ pues en principio es necesario crear un vector de dimensión mínima $|V|$, y luego aplicar las distintas funciones de hash para cada arista de E según corresponda (bajo un análisis amortizado, donde se va insertando elementos de forma dinámica y el rehashing ocurre pocas veces).

Para determinar si dos vértices v y w son adyacentes, entonces debo aplicar la función de hash $h_v(w)$ (o $h_w(v)$) para conocer si es una arista válida. Esto, en el caso en el que no haya colisiones, puede realizarse en $O(1)$. Sin embargo, si la función de hash no es muy buena, y asigna varias aristas en un mismo espacio (pensando implementación en cadena), entonces podríamos estar hablando de un costo en el peor caso de $O(|E|)$.

En cuanto a recorrer el vecindario $N(v)$ de un vértice dado v , debo recorrer la tabla de hash, que si es dinámica, entonces el tamaño es proporcional a la cantidad de vecinos, es decir, se tiene un costo temporal $O(|N(v)|)$.

Insertar un vértice v con su conjunto de vecinos $N(v)$, si lo pensamos bajo un análisis amortizado (donde la inserción en el vector de vértices es $O(1)$), entonces se tiene un costo temporal de $O(|N(v)|)$ pues se debe calcular almacenar cada $w \in N(v)$ en la tabla de hash de v y a v en la tabla de hash de w (siguiendo la hipótesis de inserción dinámica y bajo nivel de rehashing).

El insertar una arista vw , si se tiene buenas funciones de hash, con factores de carga controlados, entonces se tiene que la inserción individual en las respectivas tablas de hash es $O(1)$ (y acceder a los vecindarios de cada arista es también $O(1)$).

Si se quiere remover un vértice v con todas sus adyacencias, entonces suponiendo que no se realiza redimensionamiento del vector de vértices (sino que simplemente la información se limpia/olvida), entonces el costo temporal recae en los tiempos que lleva la eliminación de las aristas wv para cada $w \in N(v)$. Si las funciones de hash son buenas, entonces la búsqueda y remoción presentan costos $O(1)$; y luego el costo temporal final es $O(|N(v)|)$.

En cambio, si solo se desea remover una única arista vw , se lo puede hacer en $O(1)$ si se cumplen las hipótesis anteriormente mencionadas.

Finalmente, mantener un orden de $N(v)$ de acuerdo a algún invariante que permita recorrer cada vecindario en un orden dado tampoco es viable en las tablas de hash, donde los lugares en donde se guarda cada $w \in N(v)$ se calcula a partir de fórmulas matemáticas que utiliza la función de hash h_v .

Es decir, requeriría recorrer el vecindario $N(v)$, extraer los elementos, aplicar una función de ordenamiento y finalmente guardar la información en alguna estructura auxiliar (lo cual se podría lograr en un costo de $O(|N(v)|\log(|N(v)|))$).

Ejercicio 15: *Adyacencia Eficiente*

Demostrar que las representaciones por listas de adyacencias de un grafo (ejercicio anterior) se pueden construir en $O(n + m)$ tiempo. ¿Qué ocurre si se usa una tabla de hash? ¿Y si se construye una matriz de adyacencias?

Solución por Lista de Adyacencias

1. Inicializo en $O(n)$ un vector donde cada posición v representa $N(v)$, la cual es una lista inicializa vacía (lo que toma $O(1)$).
2. Para cada $(v, w) \in E$, inserto v en $N(w)$ y w en $N(v)$. Acceder a $N(v)$, $N(w)$ es $O(1)$ (pues es simplemente dirigirse en el vector de vértices a las posiciones v, w respectivamente) y agregar un elemento al final de una lista toma $O(1)$. En conjunto (es decir, considerando la totalidad de aristas), estas operaciones se realizan en $O(m)$

Por lo tanto: Costo Temporal Final = $O(n) + O(m) = O(n + m)$ ✓

Es decir, como la representación por listas de adyacencia requiere visitar cada vértice una vez y procesar cada arista una cantidad constante de veces, el tiempo de construcción es lineal respecto al tamaño de la entrada ($n + m$).

Solución por Tabla de Hash

1. Inicializo en $O(n)$ un vector donde cada posición v representa $N(v)$, la cual es una tabla de hash, que puedo inicializar de un tamaño k inicial.
2. Para cada $(v, w) \in E$, inserto v en $N(w)$ y w en $N(v)$. Acceder a $N(v)$, $N(w)$ es $O(1)$ (pues es simplemente dirigirse en el vector de vértices a las posiciones v, w respectivamente) y si la función de hash es buena (evita colisiones excesivas) y si pensamos en inserciones dinámicas con un bajo nivel de rehashing (factores de carga controlados), entonces en un análisis amortizado las inserciones cuestan $O(1)$. Luego, en conjunto (tomando la totalidad de las aristas de E), las operaciones tienen un costo amortizado de $O(m)$.

Por lo tanto, Costo Temporal Final = $O(n) + O(m) = O(n + m)$

Solución por Matriz de Adyacencia

1. Inicializo en $O(n^2)$ una matriz A donde cada vector A_v contiene valores booleanos que definen para cada $w \in V \setminus \{v\}$ representa si son o no vecinos v y w . Generalmente, la matriz, en un inicio, se la inicia con el valor *False* en cada una de las celdas.
2. Para cada $(v, w) \in E$, debo asignar el valor *True* a las casillas $A_v[w]$ y $A_w[v]$. Dado que, individualmente, esto presenta un costo $O(1)$, luego en conjunto, el costo es $O(m)$.

Por lo tanto, Costo Temporal Final = $O(n^2) + O(m) = O(n^2)$
(Esto es pues $|E| = m \leq n^2 = |V|^2$)

Ejercicio 17: *Cazador de Ciclos*

En este ejercicio diseñamos un algoritmo para encontrar ciclos en un digrafo. Decimos que un digrafo es *acíclico* cuando no tiene ciclos dirigidos. Recordar que un (di)grafo es *trivial* cuando tiene un sólo vértice.

- a) Demostrar con un argumento constructivo que si todos los vértices de un digrafo D tienen grado de salida mayor a 0, entonces D tiene un ciclo.
- b) Diseñar un algoritmo que permita encontrar un ciclo en un digrafo D cuyos vértices tengan todos grado de salida mayor a 0.
- c) Explicar detalladamente (sin usar código) cómo se implementa el algoritmo del inciso anterior. El algoritmo resultante tiene que tener complejidad temporal $O(n + m)$.
- d) Demostrar que un digrafo D es acíclico si y solo si D es trivial o D tiene un vértice con $d_{out}(v) = 0$ tal que $D \setminus \{v\}$ es acíclico.
- e) A partir del inciso anterior, diseñar un algoritmo que permita determinar si un grafo D tiene ciclos. En caso negativo, el algoritmo debe retornar una lista $v_1, \dots, v_n$ de vértices tales que $d_{out}(v_i) = 0$ en $D \setminus \{v_1, \dots, v_{i-1}\}$ para todo i . En caso afirmativo, el algoritmo debe retornar un ciclo.
- f) Explicar detalladamente (sin usar código) cómo se implementa el algoritmo del inciso anterior. El algoritmo resultante tiene que tener complejidad temporal $O(n + m)$.

Solución ítem a)

Veremos que se cumple la propiedad para toda cantidad de vértices $n \in \mathbb{N}_{\geq 3}$.

Caso Base: $n = 2$

Luego, sea D el digrafo tal que $V = \{v_0, v_1\}$ y $d_{out}(v_0), d_{out}(v_1) > 0$.

Necesariamente se tiene que $D = (V, E)$ con $E = \{(v_0, v_1), (v_1, v_0)\}$.

Esto implica que existe camino de ida y vuelta entre v_0 y v_1 , ya sea si quiero $v_0 \rightarrow v_1$ tengo la arista (v_0, v_1) ; o bien existe camino en sentido $v_1 \rightarrow v_0$ pues $(v_1, v_0) \in E$.

Así, se tiene el ciclo $v_0 \rightarrow v_1 \rightarrow v_0$ ✓

Caso Base: $n=3$

Sea D un digrafo de tres vértices, $V = \{v_0, v_1, v_2\}$, donde $d_{out}(v_i) > 0$

$\forall 0 \leq i \leq 2$ (es decir, $|E| \geq 3$).

Luego, si $\exists 0 \leq i < j \leq 2$ tal que $(v_i, v_j), (v_j, v_i) \in E$ entonces $v_i \rightarrow v_j \rightarrow v_i$ es un ciclo ✓

Sin embargo, caso contrario, entonces si tomo v_0 , dado que $d(v_0) > 0$, entonces:

$$(v_0, v_1) \in E \quad \vee \quad (v_0, v_2) \in E$$

SPG, tomo $(v_0, v_1) \in E$ (caso $(v_0, v_2) \in E$ es análogo). Luego, como por suposición $(v_1, v_0) \notin E$ entonces $(v_1, v_2) \in E$. Nuevamente, dado que por suposición $(v_2, v_1) \notin E$, entonces eso implica que $(v_2, v_0) \in E$.

Por lo tanto, se forma $v_0 \rightarrow v_1 \rightarrow v_2 \rightarrow v_0$, el cual es un ciclo. ✓

Caso general: sea $n \in \mathbb{N}_{\geq 3}$

Sea D un digrafo de n vértices, $V = \{v_0, \dots, v_{n-1}\}$, donde para cada vértice $v_i \in V$ se tiene que $d(v_i) > 0$.

Si $\exists 0 \leq i < j < n$ tal que $(v_i, v_j) \in E \wedge (v_j, v_i) \in E$ entonces el grafo presenta el ciclo $v_i \rightarrow v_j \rightarrow v_i$ ✓.

Supongamos ahora que $\forall 0 \leq i, j < n$ con $i \neq j$, si $(v_i, v_j) \in E \Rightarrow (v_j, v_i) \notin E$.

Sea entonces $v_{i_0} = v_0$. Como $d(v_{i_0}) > 0$, existe $v_{i_1} \in V \setminus \{v_0\}$ con $(v_{i_0}, v_{i_1}) \in E$.

De manera análoga, como $d(v_{i_1}) > 0$ y por suposición $(v_{i_1}, v_{i_0}) \notin E$, entonces existe $v_{i_2} \in V \setminus \{v_{i_0}, v_{i_1}\}$ tal que $(v_{i_1}, v_{i_2}) \in E$.

Luego, si $(v_{i_2}, v_{i_0}) \in E \Rightarrow v_{i_0} \rightarrow v_{i_1} \rightarrow v_{i_2} \rightarrow v_{i_0}$ es un ciclo ✓

Si es que no, entonces existe $v_{i_3} \in V \setminus \{v_{i_0}, v_{i_1}, v_{i_2}\}$ tal que $(v_{i_2}, v_{i_3}) \in E$. Luego, si $(v_{i_3}, v_{i_j}) \in E$ para algún $0 \leq j < 3 - 1$ entonces se forma

$$v_{i_j} \rightarrow v_{i_{j+1}} \rightarrow \dots \rightarrow v_{i_3} \rightarrow v_{i_2} \rightarrow v_{i_1} \rightarrow v_{i_0} \rightarrow v_{i_j} \quad \text{el cual es un ciclo} \quad \checkmark$$

Si repetimos esto de manera sucesiva, es decir para cada $2 \leq k < n - 1$, si existe v_{i_j} tal que $(v_{i_k}, v_{i_j}) \in E$ con $0 \leq j < k - 1$ entonces se obtiene el ciclo

$$v_{i_j} \rightarrow v_{i_{j+1}} \rightarrow \dots \rightarrow v_{i_k} \rightarrow v_{i_j}.$$

Si es que no, entonces como $d(v_{i_k}) > 0$ entonces existe $v_{i_{k+1}} \in V \setminus \{v_{i_0}, \dots, v_{i_k}\}$ tal que $(v_{i_k}, v_{i_{k+1}}) \in E$; y repetimos el proceso.

Si $k = n - 1$, entonces necesariamente existe v_{i_j} con $0 \leq j < n - 1$ tal que $(v_{i_k}, v_{i_j}) \in E$.

Y es entonces que $v_{i_j} \rightarrow v_{i_{j+1}} \rightarrow \dots \rightarrow v_{i_{n-1}} \rightarrow v_{i_j}$ es un ciclo ✓
 Luego, como a partir de un digrafo D de n vértices con grados de salida mayor a 0 cualquiera, demostré que D tiene un ciclo; entonces vale para todo grafo D de n vértices con grados de salida mayor a 0.
 Finalmente, como ví los casos base donde la cantidad de vértices es $n \in \{2, 3\}$ y caso general con $n \in \mathbb{N}_{\geq 3}$, entonces la propiedad vale $\forall n \in \mathbb{N}_{\geq 2}$ como se quería probar.



Solución ítem b)

Planteemos un algoritmo para hallar un ciclo en un digrafo D donde sus vértices tengan todos grados de salida mayor a 0.

Entrada: V, E

- 1) Inicializo una lista C ordenada, con un elemento $v_{actual} \in V$ cualquiera.
- 2) Mientras no haya encontrado un ciclo:
 - Observo los vecinos de v_{actual}
 - Si $C = \{v_{i_0}, \dots, v_{actual}\}$ y $\exists 0 \leq j < |C|$ tal que $(v_{actual}, v_{i_j}) \in E$ entonces formo el ciclo $v_{i_j} \rightarrow v_{i_{j+1}} \rightarrow \dots \rightarrow v_{actual} \rightarrow v_{i_j}$
 - Caso contrario, añado $u \in N(v_{actual})$ cualquiera; y luego nos movemos al mismo (u es nuestro nuevo vértice actual).

Por lo demostrado en el ítem a), este algoritmo necesariamente devolverá un ciclo. Esto es pues si se alcanza el punto en el que $|C| = n$, entonces como $d(v_{i_{|C|-1}}) > 0$ (donde $v_{i_{|C|-1}} = v_{actual}$), debe existir $v \in C$ tal que $(v, v_{i_{|C|-1}}) \in E$, y por lo tanto poder así generar el ciclo de la forma que el algoritmo describe.

Solución ítem c)

- 1) Sea L una lista de adyacencias del grafo D . Es decir, L es un vector de tamaño $|V|$ donde cada posición v de L es una lista de vecinos de v ($N(v)$). Por lo visto en el [Ejercicio 15](#), esto toma $O(n + m)$.
- 2) Inicializo C un vector de tamaño $|V|$ y pongo en su primer posición v_0 . Denomino *actual* a la última posición ocupada en C . Adicionalmente, inicializo $\tilde{C}$ de tamaño $|V|$ un vector donde cada posición es *False*, excepto la posición 0 (correspondiente a v_0), la cual será *True*. Esto tiene un costo temporal de $O(n)$.

3) Mientras no se haya encontrado un ciclo:

- Si para algún $u \in N(v_{actual})$, $\tilde{C}[u] = True$ entonces formo el ciclo que empieza desde u , avanza en C hasta v_{actual} y finaliza en u (debiendo revisar en qué posición de C se encuentra; para formar el ciclo con todos los elementos que se encuentran a partir de ese u). Es decir, defino $res = \{u, \dots, v_{actual}, u\}$.
- Caso contrario, añado algún u (el primero o el último por ejemplo) a C , actualizo $actual$ y asigno $\tilde{C}[u] = True$.

Como a lo sumo la cantidad total de aristas revisadas en la totalidad del trayecto es m , el costo temporal en el peor caso es de $O(m)$.

Luego, hallar en qué posición se encuentra el primer (y último) vértice de mi ciclo y pasar la información de C a res cuesta en el peor caso $O(n)$ (y se realiza una única vez). Por lo tanto, se trata de un costo temporal $O(n+m)$.

Finalmente: Costo Temporal Final = $O(n+m)$.

Solución ítem d)

$\Rightarrow$) Como D es acíclico, entonces o bien D es trivial, o luego $\exists v \in V_D$ tal que $d_{out}(v) = 0$. Caso contrario, por [inciso a](#)), D tendría un ciclo.

Luego, nótese que si llamamos $\tilde{D}$ al grafo inducido por $D \setminus \{v\}$, éste es también acíclico. De lo contrario, es decir, si $\tilde{D}$ tiene un ciclo $v_{i_0} \rightarrow v_{i_1} \rightarrow \dots \rightarrow v_{i_{|C|-1}}$ entonces como $E_{\tilde{D}} \subseteq E_D$ entonces $v_{i_0} \rightarrow \dots \rightarrow v_{i_{|C|-1}}$ es también un ciclo en D . Esto nos lleva a un *ABS* pues D acíclico.

Por lo tanto, si D es acíclico, entonces o bien D es trivial o luego $\exists v \in D : d_{out}(v) = 0$ y $D \setminus \{v\}$ es acíclico. ✓

$\Leftarrow$) Si D es el digrafo trivial, entonces D es acíclico.

Por otro lado, si $\exists v \in D$ con $d_{out}(v) = 0$ tal que $D \setminus \{v\}$ es acíclico, eso quiere decir que, dado que $d_{out}(v) = 0$, $\nexists C_D$ ciclo del grafo D tal que $v \in C$; puesto que $\forall (u, \tilde{u}) \in E_D, u \neq v \quad (v \nrightarrow \tilde{u} \text{ para ningún } \tilde{u} \in V_D)$.

Y como $D \setminus \{v\}$ es acíclico, eso quiere decir que $\nexists C_D$ ciclo del grafo D tal que $v \notin C$. Caso contrario, C sería un ciclo de $D \setminus \{v\}$ (pues $v \notin C$) lo cual es *ABS* pues $D \setminus \{v\}$ es acíclico.

Luego, como $\nexists C_D$ ciclo en el digrafo D tal que $v \in C_D \vee v \notin C_D$ entonces D es acíclico.

Por lo tanto, si D es trivial, o si $\exists v \in V_D$ con $d_{out}(v) = 0$ tal que $D \setminus \{v\}$ es acíclico, entonces D es acíclico. ✓

Solución ítem e)

- 1) Inicializo A un vector vacío.
- 2) Mientras exista $v \in V_D \setminus A$ tal que $d_{out}(v) = 0$ en $D \setminus A$, agrego v a A .
- 3) Si $D \setminus A$ es vacío, devuelvo A . Caso contrario, devuelvo el resultado de aplicar el algoritmo del [inciso b\)](#) al digrafo $D \setminus A$.

Solución ítem f)

- 1) Sea L es un vector donde cada posición v es una lista de vértices u tales que $(u, v) \in E$. Esto, por un razonamiento similar al de [Ejercicio 15](#), convella un costo temporal $O(n + m)$.
- 2) Sea también G un vector de tamaño $|V_D|$ donde cada posición v denota $d_{out}(v)$; y sea A un vector de tamaño $|V_D|$ con *actual* un iterador que indica la última posición ocupada en A (*actual* es inicializada en -1). La creación de G convella un costo $O(n + m)$ (pues es la creación de un vector de tamaño n , más la realización de operaciones $O(1)$ como sumas y asignaciones por cada $e \in E$). Por último, sea B un vector de booleanos de tamaño $|V_D|$ que cada posición v describe si $v \in A$ (en principio, $B[v] = False$ para todo $v \in V_D$).
- 3) Recorro G una única vez para hallar aquellos vértices tales que su grado de salida es 0 (lo que cuesta $O(n)$). Los agrego en los primeros espacios libres de A , actualizo *actual* en función de la cantidad de vértices agregados (en $O(1)$). A la vez, asigno *True* a los espacios correspondientes de B (nuevamente, en $O(1)$).
- 4) Inicializo un iterador a en 0 que recorre los elementos de A . Mientras que $a \leq actual$, para cada $v \in L[A[a]]$, resto 1 a $G[v]$. Si $G[v]$ es 0, adiciono v a A , actualizo *actual* y asigno *True* a $B[v]$. Cualquiera sea el caso, actualizo a . Esto en el peor caso presentará una complejidad $O(n + m)$ pues implicaría que *actual* alcanzó el valor $|V_D| - 1$; y que recorrí $L[v] \ \forall v \in V_D$, lo cual es en conjunto $|E|$.
- 5) Si $a = actual = |V_D| - 1$, entonces devuelvo A .
Caso contrario, creo un vector $\tilde{V}$ con las posiciones v de B tal que $B[v] = False$. Luego, creo un vector de tuplas $\tilde{E}$ donde para cada $(u, \tilde{u}) \in E$, si $B[u] = False \wedge B[\tilde{u}] = False$, entonces $(u, \tilde{u}) \in \tilde{E}$. En conjunto, esto presenta un costo temporal de $O(n + m)$.
Y, finalmente, aplico el algoritmo del [inciso c\)](#) a $\tilde{V}, \tilde{E}$; lo que, como se demostró, se logra en $O(n + m)$.